



NATIONAL UNIVERSITY
of Computer & Emerging Sciences

MUHAMMAD ALI HARIS

22K-4239

CN ASSIGNMENT#02

CODE SNIPPETS:

```
EXPLORER  ...  rdt3.py  x  sr.py  gbn.py  main.py  network_simulator.py  fsm.py  1  config.py

CN ASSIGN 02
  > __pycache__
  > fsm_outputs
  config.py
  fsm.py  1
  gbn.py
  main.py
  network_simulator.py
  rdt3.py
  sr.py

rdt3.py  >  rdt_send
5  def rdt_send(data):
6      seq = 0
7      for item in data:
8          while True:
9              pkt = {'seq': seq, 'data': item}
10             print(f"Sending: {pkt}")
11             result = simulate_network(pkt)
12
13             start = time.time()
14             while time.time() - start < TIMEOUT:
15                 if result and result['data'] != 'CORRUPTED':
16                     print(f"ACK received for seq {seq}")
17                     seq += 1
18                     break
19                 time.sleep(0.5)
20             else:
21                 print("Timeout. Retrying...")
22                 continue
23             break
```

```
EXPLORER  ...  rdt3.py  x  sr.py  gbn.py  main.py  network_simulator.py  fsm.py  1  config.py

CN ASSIGN 02
  > __pycache__
  > fsm_outputs
  config.py
  fsm.py  1
  gbn.py
  main.py
  network_simulator.py
  rdt3.py
  sr.py

sr.py  >  sr_send
5  def sr_send(data):
30     print(f"Resending: {pkt}")
31     result = simulate_network(pkt)
32     if result and result['data'] != 'CORRUPTED':
33         print(f"ACK received for seq {seq}")
34         acked[seq] = True
35         timers[seq] = time.time()
36
37     while base in acked and acked[base]:
38         del window[base]
39         del acked[base]
40         del timers[base]
41         base += 1
42
```

```
EXPLORER  ...  rdt3.py  sr.py  gbn.py  main.py  network_simulator.py  fsm.py 1  config.py  ▾
└─ CN ASSIGN 02
  > __pycache__
  > fsm_outputs
  config.py
  fsm.py 1
  gbn.py
  main.py
  network_simulator.py
  rdt3.py
  sr.py

gbn.py > gbn_send
5  def gbn_send(data):
10 while base < len(data):
11     while next_seq < base + WINDOW_SIZE and next_seq < len(data):
12         pkt = {'seq': next_seq, 'data': data[next_seq]}
13         print(f"Sending: {pkt}")
14         result = simulate_network(pkt)
15         window[next_seq] = result
16         timers[next_seq] = time.time()
17         next_seq += 1
18
19     time.sleep(1)
20     current_time = time.time()
21
22     for seq in list(window):
23         if seq == base and window[seq] and window[seq]['data'] != 'CORRUPTED':
24             print(f"ACK received for seq {seq}")
25             del window[seq]
26             del timers[seq]
27             base += 1
28         elif seq == base and current_time - timers[seq] > TIMEOUT:
29             print(f"Timeout at seq {seq}, resending window from {base}")
30             for i in range(base, next_seq):
31                 pkt = {'seq': i, 'data': data[i]}
32                 print(f"Resending: {pkt}")
33                 result = simulate_network(pkt)
34                 window[i] = result
35                 timers[i] = time.time()
36             break
37
```

```
EXPLORER  ...  rdt3.py  sr.py  gbn.py  main.py  network_simulator.py  fsm.py 1  config.py  ▾
└─ CN ASSIGN 02
  > __pycache__
  > fsm_outputs
  config.py
  fsm.py 1
  gbn.py
  main.py
  network_simulator.py
  rdt3.py
  sr.py

main.py > ...
1  from rdt3 import rdt_send
2  from gbn import gbn_send
3  from sr import sr_send
4  from fsm import generate_rdt3_fsm, generate_gbn_fsm, generate_sr_fsm
5
6  data = ["pkt1", "pkt2", "pkt3"]
7
8  print("\n----- RDT 3.0 -----")
9  rdt_send(data)
10
11 print("\n----- Go-Back-N -----")
12 gbn_send(data)
13
14 print("\n----- Selective Repeat -----")
15 sr_send(data)
16
17 generate_rdt3_fsm()
18 generate_gbn_fsm()
19 generate_sr_fsm()
20
```

```
EXPLORER
...
rdt3.py
sr.py
gbn.py
main.py
network_simulator.py x
fsm.py 1
config.py

CN ASSIGN 02
> __pycache__
> fsm_outputs
config.py
fsm.py 1
gbn.py
main.py
network_simulator.py
rdt3.py
sr.py

network_simulator.py > simulate_network
1 import random
2 import time
3 from config import LOSS_PROB, CORRUPT_PROB, DELAY_PROB
4
5 def simulate_network(packet):
6     if random.random() < LOSS_PROB:
7         print("Packet lost.")
8         return None
9
10    if random.random() < CORRUPT_PROB:
11        print("Packet corrupted.")
12        packet['data'] = 'CORRUPTED'
13
14    if random.random() < DELAY_PROB:
15        delay = round(random.uniform(0.5, 1.5), 2)
16        print(f"Packet delayed by {delay}s")
17        time.sleep(delay)
18
19    return packet
```

```
EXPLORER
...
rdt3.py
sr.py
gbn.py
main.py
network_simulator.py
fsm.py 1 x
config.py

CN ASSIGN 02
> __pycache__
> fsm_outputs
config.py
fsm.py 1
gbn.py
main.py
network_simulator.py
rdt3.py
sr.py

fsm.py > generate_rdt3_fsm
1 from graphviz import Digraph
2 import os
3
4 os.makedirs("fsm_outputs", exist_ok=True)
5
6 def generate_rdt3_fsm():
7     dot = Digraph("rdt3")
8     dot.attr(rankdir="LR")
9     dot.edge('Wait0', 'ACK0', 'send pkt0')
10    dot.edge('ACK0', 'Wait1', 'recv ACK0')
11    dot.edge('ACK0', 'ACK0', 'timeout/NAK0')
12    dot.edge('Wait1', 'ACK1', 'send pkt1')
13    dot.edge('ACK1', 'Wait0', 'recv ACK1')
14    dot.edge('ACK1', 'ACK1', 'timeout/NAK1')
15    dot.render('fsm_outputs/rdt3_fsm', format='png', cleanup=True)
16
17
18 def generate_gbn_fsm():
19     dot = Digraph("GBN")
20     dot.attr(rankdir="LR")
21     dot.edge('Send', 'WaitACK', 'send pkt(s)')
22     dot.edge('WaitACK', 'Send', 'recv ACK (base updated)')
23     dot.edge('WaitACK', 'WaitACK', 'timeout or partial ACK')
24     dot.render('fsm_outputs/gbn_fsm', format='png', cleanup=True)
25
26
27 def generate_sr_fsm():
28     dot = Digraph("SR")
29     dot.attr(rankdir="LR")
30     dot.edge('Send', 'WaitACKs', 'send pkt(s)')
31     dot.edge('WaitACKs', 'WaitACKs', 'recv individual ACK or timeout')
32     dot.render('fsm_outputs/sr_fsm', format='png', cleanup=True)
```

```
EXPLORER
...
rdt3.py
sr.py
gbn.py
main.py
network_simulator.py
fsm.py 1
config.py x

CN ASSIGN 02
> __pycache__
> fsm_outputs
config.py
fsm.py 1
gbn.py
main.py
network_simulator.py
rdt3.py
sr.py

config.py > ...
1 PACKET_SIZE = 1024
2 TIMEOUT = 3
3 WINDOW_SIZE = 4
4 LOSS_PROB = 0.2
5 CORRUPT_PROB = 0.2
6 DELAY_PROB = 0.1
```

OUTPUTS AND FSMS

RDT 3.0

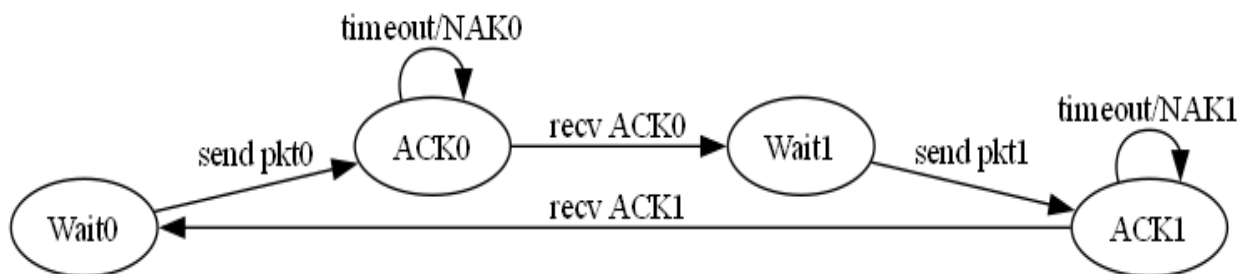
```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Saad Haris\OneDrive\Desktop\CN Assign 02> python main.py

----- RDT 3.0 -----

Sending: {'seq': 0, 'data': 'pkt1'}
ACK received for seq 0
Sending: {'seq': 1, 'data': 'pkt2'}
Packet corrupted.
Timeout. Retrying...
Sending: {'seq': 1, 'data': 'pkt2'}
Packet corrupted.
Timeout. Retrying...
Sending: {'seq': 1, 'data': 'pkt2'}
Packet lost.
Timeout. Retrying...
Sending: {'seq': 1, 'data': 'pkt2'}
ACK received for seq 1
Sending: {'seq': 0, 'data': 'pkt3'}
Packet lost.
Timeout. Retrying...
Sending: {'seq': 0, 'data': 'pkt3'}
ACK received for seq 0
```

FSM:



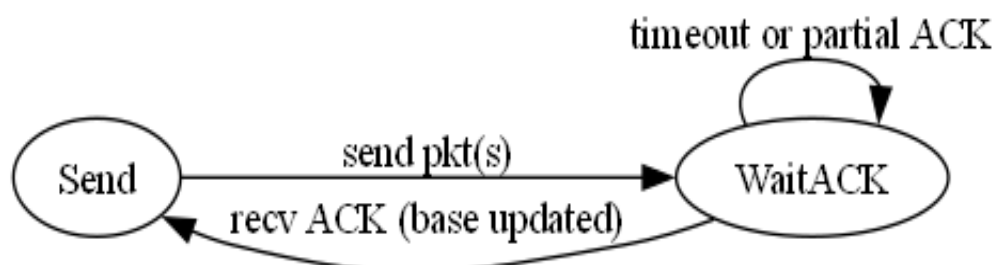
GO-BACK-N

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

----- Go-Back-N -----

Sending: {'seq': 0, 'data': 'pkt1'}
Packet corrupted.
Sending: {'seq': 1, 'data': 'pkt2'}
Packet corrupted.
Sending: {'seq': 2, 'data': 'pkt3'}
Packet corrupted.
Timeout at seq 0, resending window from 0
Resending: {'seq': 0, 'data': 'pkt1'}
Resending: {'seq': 1, 'data': 'pkt2'}
Resending: {'seq': 2, 'data': 'pkt3'}
ACK received for seq 0
Timeout at seq 0, resending window from 0
Resending: {'seq': 0, 'data': 'pkt1'}
Resending: {'seq': 1, 'data': 'pkt2'}
Timeout at seq 0, resending window from 0
Timeout at seq 0, resending window from 0
Resending: {'seq': 0, 'data': 'pkt1'}
Timeout at seq 0, resending window from 0
Resending: {'seq': 0, 'data': 'pkt1'}
Resending: {'seq': 1, 'data': 'pkt2'}
Timeout at seq 0, resending window from 0
Resending: {'seq': 0, 'data': 'pkt1'}
Resending: {'seq': 0, 'data': 'pkt1'}
Resending: {'seq': 1, 'data': 'pkt2'}
Resending: {'seq': 2, 'data': 'pkt3'}
ACK received for seq 0
ACK received for seq 0
ACK received for seq 1
ACK received for seq 2
```

FSM:



SELECTIVE-REPEAT

```
----- Selective Repeat -----
```

```
Sending: {'seq': 0, 'data': 'pkt1'}
```

```
ACK received for seq 0
```

```
Sending: {'seq': 1, 'data': 'pkt2'}
```

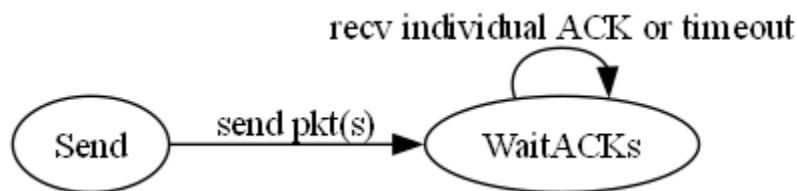
```
ACK received for seq 1
```

```
Sending: {'seq': 2, 'data': 'pkt3'}
```

```
ACK received for seq 2
```

```
PS C:\Users\Saad Harris\OneDrive\Desktop\CN Assign 02> █
```

FSM:



Some Explanation:

In **RDT**, each packet is sent only after previous pkt is acknowledged.

In **GBN**, all 3 packets are sent out because of window size containing all of them. When pkt1 lost and timeout, it's resend. Since pkt2 was lost, it is re-transmitted because window moved on.

Selective Repeat, pkt1 was lost, but the remaining packets were acknowledged. After timeout, ONLY pkt1 is resent and acknowledged.

